Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky

Návrh a testování prostředí pro AI agenty v softwarovém vývoji

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Autor: Thanh An Nguyen
Vedoucí práce: Ing. Jiří Korčák

Praha, květen 2026

Prohlášení

Prohlašuji, že jsem bakalářskou práci zpracoval samostatně a že jsem uvedl všechny použité prameny a literaturu, ze kterých jsem čerpal.

V Praze dne 1. března 2026

Thanh An Nguyen

Poděkování

Poděkování.

Abstrakt

TODO: Napsat až budou hotové výsledky.

Klíčová slova

AI agenti, softwarový vývoj, prostředí, scaffolding, kvalita kódu

Abstract

TODO: Write after results are complete.

Keywords

AI agents, software development, environment, scaffolding, code quality

Obsah

Úvod **10**

1 Vymezení problému a cílů práce **11**
1.1 Motivace . . . 11
1.2 Cíle práce . . . 11
1.3 Rozsah práce . . . 12

2 Teoretická východiska **13**
2.1 Softwarové inženýrství a životní cyklus . . . 13
2.1.1 Definice a komplexita software . . . 13
2.1.2 Fáze životního cyklu . . . 13
2.1.3 Modely a metodiky . . . 13
2.2 AI coding agenti . . . 13
2.2.1 Základní pojmy . . . 13
2.2.2 Typy coding agentů . . . 13
2.2.3 Jak agenti mění SDLC . . . 13
2.2.4 Hodnocení agentů a benchmarky . . . 13
2.3 Scaffolding a instrukce pro agenty . . . 13
2.3.1 Problém kontextu . . . 13
2.3.2 Přístupy k memory a kontextu . . . 13
2.3.3 Struktura instrukcí . . . 13
2.4 Měření kvality software . . . 13
2.4.1 Klasifikace softwarových metrik . . . 13
2.4.2 Testování a mutation testing . . . 13
2.4.3 Statická analýza a složitost kódu . . . 13
2.4.4 LLM-as-judge . . . 13

3 Metodika **14**
3.1 Návrhový výzkum (DSR) . . . 14
3.2 Sada metrik . . . 15
3.2.1 Procesní metriky (P1–P2) . . . 15
3.2.2 Produktové metriky (Q1–Q8) . . . 16
3.2.3 Metriky efektivity (E1–E3) . . . 17
3.2.4 Aplikace LLM-as-judge . . . 18
3.3 Experimentální design . . . 18
3.3.1 Výběr projektu . . . 18
3.3.2 Fixní proměnné . . . 18
3.3.3 Pilotní iterace . . . 19
3.3.4 Komparativní variace . . . 20
3.4 Přehledová tabulka . . . 21

3.5 Omezení a validita 21

4 Praktická část **23**

4.1 Příprava experimentu 23

4.1.1 Konstrukce specifikace 23

4.1.2 Referenční implementace 24

4.1.3 Konstrukce baseline instrukcí 24

4.2 Pilotní iterace 26

4.2.1 Pilot-r1: baseline 26

4.3 Komparativní variace 27

4.3.1 Ablace 27

4.3.2 Substituce 27

4.4 Souhrnné výsledky 27

5 Vyhodnocení a diskuse **28**

5.1 Interpretace výsledků 28

5.2 Porovnání s literaturou 28

5.3 Limity a hrozby validity 28

5.4 Doporučení pro praxi 28

5.5 Náměty pro další výzkum 28

Závěr **29**

Bibliografie **30**

A Formulář v plném znění **35**

B Zdrojové kódy výpočetních procedur **36**

C Baseline AGENTS.md **37**

Seznam obrázků

Seznam tabulek

3.1 Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ 21

4.1 Mapování sekcí baseline `AGENTS.md` na metriky 25
4.2 Pilot-r1: naměřené metriky 26

Seznam použitých zkratek

TODO Doplňte zkratky a tento řádek smažte.

Úvod

1. Vymezení problému a cílů práce

1.1 Motivace

[DRAFT]

AI agenti se používají k vývoji softwaru, ale výsledky ukazují že to není tak jednoduché jak se čekalo. Randomizovaná studie METR (METR, 2025) na 246 úlohách ukázala, že s AI nástroji byli vývojáři o 19 % pomalejší. Agent často produkuje funkční výstup, nedodržuje přitom vývojové praktiky: slučuje nesouvisející změny, přeskakuje testy, ignoruje specifikaci.

Otázka není jen jestli agent úkol vyřešil, ale jak. Existující benchmarky jako SWE-bench (Jimenez et al., 2024) měří pouze výsledek: vyřešil agent úkol, ano nebo ne. Kvalitu práce, transparentnost procesu a možnost řídit chování agenta v průběhu vývoje tyto benchmarky nepostihují. Agent může hlásit úkol jako dokončený, přestože implementace pokrývá jen část požadavků, a bez metrik procesu to není jak systematicky odhalit.

Chování agenta lze přizpůsobit trénováním modelu (fine-tuning) nebo instrukcemi v kontextovém okně (in-context learning). Shin et al. (Shin et al., 2025) ukazují, že ani jeden přístup systematicky nedominuje, explicitní instrukce s kontextem nicméně výrazně zlepšují výkon agenta na úlohách s kódem. S rostoucí velikostí kontextových oken se instrukce stávají praktičtější alternativou: jsou dostupné komukoliv, iterativně upravitelné a neomezují emergentní schopnosti modelu. Pro trénování modelů existují zavedené postupy; pro systematické navrhování a vyhodnocování instrukcí obdobný postup dosud nevznikl.

Tato práce navrhuje evaluační systém který měří nejen výsledek, ale i proces a kvalitu práce agenta, a popisuje postup jak s jeho pomocí iterativně vylepšovat instrukce.

1.2 Cíle práce

[DRAFT]

1. Navrhnout sadu metrik která měří proces a kvalitu práce AI agenta, ne jen výsledek.
2. Na případové studii demonstrovat iterativní postup pro návrh instrukcí s využitím těchto metrik.
3. Popsat pozorované tendence v chování agenta a vliv jednotlivých složek instrukcí na tyto tendence.

1.3 Rozsah práce

[DRAFT]

Evaluační systém a postup pro návrh instrukcí nelze navrhnout čistě teoreticky. Je potřeba je ověřit v praxi: ukázat, že metriky skutečně zachytí rozdíly v chování agenta, když dostane jiné instrukce, a že iterativní postup vede ke zlepšení. K tomu slouží případová studie, ve které opakovaně pouštíme agenta s různými variantami instrukcí a pokaždé měříme, co se změnilo.

Aby bylo možné vliv instrukcí měřit, musí být všechno ostatní stejné: model, nástroje, specifikace i výchozí stav repozitáře. Jedinou proměnnou jsou instrukce které agent dostane. Kdyby se měnilo víc věcí najednou, nebylo by jasné, co způsobilo změnu v chování.

Případová studie probíhá na jednom projektu: systému upomínek faktur. Jde o projekt s deterministickou logikou, kde stejný vstup vždy dá stejný výstup. To je důležité, protože výsledky lze ověřit automatizovanými testy, ne subjektivním posouzením. Agent pracuje od specifikace k implementaci, tedy od požadavků přes návrh po funkční kód a testy.

Práce neporovnává různé modely ani programovací jazyky a neporovnává agenta s lidským vývojářem. Navržený evaluační systém a iterativní postup jsou koncipovány obecně: kdokoli je může vzít a použít na svém projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Zdůvodnění volby projektu a podrobnosti experimentálního designu popisuje kapitola 3.

2. Teoretická východiska

2.1 Softwarové inženýrství a životní cyklus

2.1.1 Definice a komplexita software

2.1.2 Fáze životního cyklu

2.1.3 Modely a metodiky

2.2 AI coding agenti

2.2.1 Základní pojmy

2.2.2 Typy coding agentů

2.2.3 Jak agenti mění SDLC

2.2.4 Hodnocení agentů a benchmarky

2.3 Scaffolding a instrukce pro agenty

2.3.1 Problém kontextu

2.3.2 Přístupy k memory a kontextu

2.3.3 Struktura instrukcí

2.4 Měření kvality software

2.4.1 Klasifikace softwarových metrik

2.4.2 Testování a mutation testing

2.4.3 Statická analýza a složitost kódu

2.4.4 LLM-as-judge

3. Metodika

3.1 Návrhový výzkum (DSR)

[DRAFT]

Kapitola 1 definuje tři cíle (navrhnout sadu metrik, demonstrovat iterativní postup a popsat pozorované tendence), jejichž naplnění vyžaduje navrhovat řešení, ověřovat ho v praxi a na základě výsledků iterativně upravovat instrukce. Takový postup, kdy se navrhuje řešení a opakovaně vyhodnocuje, odpovídá návrhovému výzkumu (Design Science Research, DSR). Hevner et al. (Hevner et al., 2004) ho definují jako iterativní cyklus návrhu a vyhodnocení: řešení se navrhne, vyzkouší, a na základě výsledků upraví. Peffers et al. (Peffers et al., 2008) tento cyklus formalizují do šesti aktivit: (1) identifikace problému, (2) definice cílů řešení, (3) návrh, (4) demonstrace, (5) vyhodnocení a (6) komunikace výsledků. V naší práci odpovídá identifikace problému a cíle kapitole 1, návrh a demonstrace probíhají formou případové studie (tato kapitola a kapitola 4), vyhodnocení shrnuje kapitola 5 a komunikací je samotný text práce.

Aktivity návrh, demonstrace a vyhodnocení se v DSR opakují dokud řešení nedosáhne požadované kvality. V naší práci operacionalizujeme tento cyklus jako čtyři kroky: Run (spustit agenta), Measure (změřit výstup), Diagnose (identifikovat příčiny) a Fix (upravit instrukce). Každý průchod těmito kroky odpovídá jednomu průchodu Peffersovými aktivitami návrh → demonstrace → vyhodnocení. Podrobný popis kroků uvádí sekce 3.3.3. Tento postup má dvě fáze: pilotní iterace, kde cyklus opakujeme dokud agent nesplní stanovená kritéria, a komparativní variace, kde z fungujících instrukcí systematicky odebíráme nebo nahrazujeme jednotlivé části a měříme dopad na chování agenta.

DSR vyžaduje demonstraci a vyhodnocení navrženého řešení. Hevner et al. (Hevner et al., 2004) uvádí několik možných přístupů: řízený experiment (kontrola proměnných, statistická analýza), simulace, nebo případová studie. Řízený experiment by vyžadoval dostatečný počet běhů pro statistickou sílu, což je při ceně jednoho běhu (tisíce tokenů, desítky minut) nepraktické. Simulace nepřipadá v úvahu, protože chování agenta nelze modelovat bez skutečného spuštění. Zvolili jsme tedy případovou studii na jednom projektu (systém upomínek faktur), která umožňuje opakované běhy s různými instrukcemi a podrobnou analýzu každého běhu.

Z jednoho projektu nelze tvrdit, že výsledky platí pro všechny typy software. Cílem je ukázat, že navržená sada metrik dokáže kvantifikovat chování agenta a že iterativní postup vede k měřitelnému zlepšení. Yin (R. K. Yin, 2018) pro tento typ výzkumu používá pojem analytická generalizace: neříkáme "takhle to funguje vždy", ale "na tomto případu jsme ukázali princip, který lze ověřit na dalších projektech". Yin tento design nazývá embedded single-case: zkoumáme jeden projekt, ale v rámci něj provádíme více běhů agenta. Každý běh má vlastní instrukce, vlastní git historii a vlastní sadu naměřených metrik, takže je můžeme porovnávat

mezi sebou.

3.2 Sada metrik

[DRAFT]

Každý experimentální běh probíhá tak, že agent dostane prázdné GitHub repo, specifikaci v Issue #1 a instrukce v `AGENTS.md`, a autonomně implementuje zadaný projekt. Výstup jednoho běhu (kód, testy, git historie) měříme sadou 13 metrik. Postup běhu a experimentální design popisuje sekce 3.3.

Sada metrik pokrývá tři otázky: jak agent pracoval, co vyrobil a za jakou cenu. Fenton a Bieman (Fenton & Bieman, 2014) klasifikují měřitelné entity v softwarovém inženýrství do tří kategorií: proces, produkt a zdroje. V kontextu AI agentů dospívají k obdobnému rozdělení Yin et al. (2025) (Z. Yin et al., 2025), kteří hodnotí agentní frameworky ve třech dimenzích: effectiveness (výsledek), efficiency (průběh práce) a overhead (spotřebované zdroje). Naše sada metrik sleduje stejnou strukturu a obsahuje 13 metrik ve třech kategoriích:

P – proces (P1–P2) Dodržel agent předepsaný postup? Měříme dodržování workflow pravidel z instrukcí (P1) a kvalitu procesních výstupů jako commit messages a issue descriptions (P2).

Q – kvalita produktu (Q1–Q8) Je kód kvalitní? Měříme funkční korektnost (Q1–Q2), kvalitu testů (Q3–Q4) a udržovatelnost kódu (Q5–Q8).

E – efektivita (E1–E3) Za jakou cenu? Zaznamenáváme spotřebu tokenů (E1), čas (E2) a míru autonomie (E3).

Výběr konkrétních metrik vychází z potřeb případové studie a dostupných nástrojů; sada si neklade nárok na úplnost. Teoretické základy jednotlivých metrik (proč jsou validní, jaká je evidence) popisuje sekce 2.4. Tato sekce se zaměřuje na *jak* konkrétně měříme: jaký nástroj, s jakou konfigurací, co je vstup a co výstup. U každé metriky uvádíme v krátkosti co měří a pak podrobně jak.

3.2.1 Procesní metriky (P1–P2)

[DRAFT]

Procesní metriky (Fenton & Bieman, 2014) měří *jak* agent pracuje. Instrukce v AGENTS.md definují strukturovaný vývojový postup: agent má odvozovat práci ze specifikace, organizovat ji přes issues a branches, psát testy před implementací a dokumentovat rozhodnutí v commit messages a PR descriptions.

P1 (process compliance) je binární checklist pěti položek pozorovatelných z git logu a GitHub API. Skript kontroluje:

1. existenci issues s timestampem před prvním kódovým commitem (GitHub API: porovnání `created_at` issue vs. čas prvního commitu s kódem),
2. branch name obsahující číslo issue (git branch),
3. commit s testem předcházející implementačnímu commitu na téže branch (git log: hledání test souborů v diffu),
4. PR description obsahující odkaz na issue (GitHub API: regex na `#N` v těle PR),
5. referenční testy beze změn oproti originálu (git diff referenčních test souborů).

Každá položka je splněna (1) nebo nesplněna (0). Checklist je plně automatizovaný.

P2 (kvalita procesních výstupů) doplňuje P1 o kvalitativní rozměr. P1 zachytí *zda* agent dodržel postup, ale ne *jak kvalitně*: agent může vytvořit issue s jednou větou a commit message `fix`. P2 hodnotí popisnost commit messages (atomicita, konvenční prefix), issue descriptions (scope, acceptance criteria) a PR descriptions (co a proč, odkaz na issue). Hodnocení provádí LLM-as-judge (sekce 3.2.4).

3.2.2 Produktové metriky (Q1–Q8)

[DRAFT]

Produktové metriky (Fenton & Bieman, 2014) měří *co* agent vyrobil. Pokrývají tři oblasti: funguje implementace správně (Q1–Q2), detekují agentovy testy skutečné chyby (Q3–Q4) a je kód udržovatelný (Q5–Q8).

Funkční korektnost (Q1–Q2)

Q1 (API contract match) ověřuje, zda agentův kód dodržuje definované rozhraní. Referenční typy se importují a zkompilují proti agentovu kódu (`tsc`). Výsledek je binární: typy sedí, nebo ne. Q1 je vstupní podmínkou pro Q2: pokud agentův kód neimplementuje správné API, referenční testy nelze ani zkompilovat, a výsledek Q2 by byl nesmyslný.

Q2 (referenční test pass rate) měří kolik z referenčních testů projde na agentově implementaci. Vitest spustí 40 testů proti agentovu kódu; výsledek je počet passing testů z 40. Testy ověřují chování přes veřejné API (black-box, sekce 2.4.2), takže je lze spustit na libovolném běhu nezávisle na interní struktuře. Konstrukce referenční test suite popisuje sekce 4.1.2.

Kvalita testů (Q3–Q4)

Q3 (mutation score) měří, jestli agentovy testy skutečně detekují chyby (sekce 2.4.2). Stryker s konfigurací `--mutate 'src/**/*.ts'` a mutátory pro TypeScript (conditional, arithmetic, string, logical operators) systematicky zavádí drobné změny do zdrojového kódu. Vstup: agentovy testy + zdrojový kód. Výstup: procento zabitých mutantů z celkového počtu.

Q4 (AC coverage) měří, kolik z 24 acceptance criteria má odpovídající test v agentově test suite. Na rozdíl od Q2 (funguje implementace?) se Q4 ptá, jestli agent *testoval všechno co měl*. Judge dostane seznam 24 AC ze specifikace a agentovu test suite. Pro každé AC určí, zda existuje test který ho pokrývá. Výstup: počet pokrytých AC z 24. Hodnocení provádí LLM-as-judge (sekce 3.2.4).

Kvalita kódu (Q5–Q8)

Q5 (lint warnings) počítá varování a chyby z ESLint s fixní konfigurací (pravidla: recommended + strict TypeScript rules). Konfigurace je součástí experimentální infrastruktury, ne agentova repo — agent ji nemůže měnit. Výstup: celkový počet warnings + errors.

Q6 (typecheck errors) počítá chyby z `tsc --noEmit` ve strict mode. Doplňuje počet explicitních `any` ve zdrojových souborech (grep v `src/**/*.ts`) jako proxy pro obcházení typového systému.

Q7 (složitost kódu) měří cyklomatickou složitost per funkce. ESLint `complexity` rule reportuje funkce překračující nastavený práh. Výstup: maximální složitost across all functions. Práh ≤ 10 per funkce vychází z McCabe (sekce 2.4.3).

Q8 (design quality) hodnotí aspekty které automatizované nástroje nezachytí: pojmenování, oddělení zodpovědností, idiomatický TypeScript, kvalita dokumentace a zbytečná komplexita. Hodnocení provádí LLM-as-judge (sekce 3.2.4).

3.2.3 Metriky efektivity (E1–E3)

[DRAFT]

Metriky efektivity měří zdroje spotřebované při vývoji. Nemají exit kritérium a slouží k porovnání nákladů mezi iteracemi.

E1 (tokeny) počítá vstupní a výstupní tokeny z exportu agentní session. Sledování E1 motivuje zjištění Gloaguen et al. (Gloaguen et al., 2025), že context files zvyšují inference cost o více než 20 %.

E2 (trvání) měří wall-clock time v minutách od spuštění agenta po poslední commit.

E3 (dokončení) zaznamenává, zda agent dokončil úkol bez pádu, a počet restartů autocontinue pluginu jako proxy pro autonomii.

S jedním během per variaci jsou tyto hodnoty deskriptivní, ne inferenční.

3.2.4 Aplikace LLM-as-judge

[DRAFT]

Tři metriky (P2, Q4, Q8) hodnotí vlastnosti které nelze extrahovat automatizovaným nástrojem. Pro jejich hodnocení používáme metodu LLM-as-judge (sekce 2.4.4).

Volba modelu. Jako judge používáme GLM-5 (Zhipu AI). Agentní běhy provádí model Minimax, judge je tedy z jiné modelové rodiny, čímž se eliminuje self-preference bias (Panickssery et al., 2024).

Škála. Hodnocení probíhá na škále 1–3 per dimenze (poor / acceptable / good). Třístupňová škála snižuje šum při malém počtu běhů oproti jemnějšímu rozlišení.

Rubrika. Judge dostane zdrojový kód a git historii agentova běhu spolu s rubrikou obsahující kritéria pro každý stupeň. Prompt je fixní across all runs. Rubriky jsou uvedeny v příloze.

Validace. Autor nezávisle ohodnotí stejné běhy na stejné škále. Výsledky porovnáme Cohenovým κ a percent agreement.

3.3 Experimentální design

3.3.1 Výběr projektu

[DRAFT]

Případová studie potřebuje projekt, na kterém lze opakovaně spouštět agenta s různými instrukcemi a objektivně měřit výsledky. Projekt musí mít deterministickou logiku (stejný vstup vždy dá stejný výstup), aby bylo možné ověřit korektnost automatizovanými testy a mutation testingem. Zároveň musí být dostatečně malý pro více experimentálních běhů, ale obsahovat reálné nuance (hraniční případy, doménová pravidla), aby měl agent co řešit.

Zvolili jsme systém upomínek faktur: systém pro automatické odesílání připomínek k nezaplaceným fakturám. Obsahuje stavový automat (nová, po splatnosti, upomínaná, eskalovaná), časové výpočty (pracovní dny, ochranné lhůty) a pravidla pro eskalaci. Specifikace definuje 24 acceptance criteria, API kontrakt (TypeScript typy a signatury) a doménový glossary. Tato kombinace deterministické logiky a hraničních případů (víkendy, svátky, grace periods) umožňuje objektivně testovat jak funkční korektnost (Q1, Q2), tak kvalitu testů (Q3, Q4).

3.3.2 Fixní proměnné

[DRAFT]

Aby bylo možné měřit vliv instrukcí, musí být všechno ostatní konstantní. Jedinou proměnnou mezi běhy je obsah souboru `AGENTS.md`, procedurální instrukční sady, která definuje pracovní postup, omezení a quality gates. Na rozdíl od generických context files, které typicky popisují adresářovou strukturu a build příkazy (Gloaguen et al., 2025), jde o fokusované instrukce s konkrétními procedurami (Li et al., 2025).

Fixní proměnné jsou:

Prázdné GitHub repo obsahuje pouze `AGENTS.md`, konfiguraci agenta a auto-continue plugin. Žádný existující kód, testy ani `package.json`. Agent musí inicializovat projekt a zvolit strukturu sám.

Specifikace v GitHub Issue #1 obsahuje 24 acceptance criteria, API kontrakt (TypeScript typy a signatury), doménový glossary a out of scope. Jediný vstup který agent dostane kromě instrukcí.

Auto-continue plugin je hook který detekuje kdy se agent zastaví a automaticky ho restartuje. Počítadlo restartů a kontrola otevřených issues zajišťují běh bez manuálního zásahu. Metrika E3 z toho čerpá.

Model Minimax-2.5 byl zvolen jako model schopný autonomně dokončit zadaný úkol. Cílem práce je vyhodnotit sadu metrik a iterativní postup, ne srovnávat výkon modelů — volba konkrétního modelu je v tomto ohledu sekundární. Minimax je zároveň z jiné modelové rodiny než judge (GLM-5, Zhipu AI), čímž se eliminuje self-preference bias (sekce 2.4.4).

System prompt agenta (`build.md`) nahrazuje výchozí system prompt nástroje OpenCode, jehož instrukce o verzování konfliktvaly s procesními požadavky experimentu. Vlastní system prompt obsahuje pouze obecné kódové konvence (styl, bezpečnost, zákaz komentářů) a odkaz na `AGENTS.md`. Veškeré procesní instrukce jsou výhradně v `AGENTS.md`, aby procesní chování agenta bylo řízeno jedinou proměnnou experimentu.

3.3.3 Pilotní iterace

[DRAFT]

Pilotní fáze aplikuje DSR build-evaluate cyklus na instrukce. Každá iterace má čtyři kroky:

1. **Run.** Skript vytvoří prázdné GitHub repo, zkopíruje aktuální `AGENTS.md`, vytvoří Issue #1 se specifikací a spustí agenta. Session transcript se exportuje po dokončení.
2. **Measure.** Skript extrahuje metriky ze tří zdrojů: git log a GitHub API (P1), testy a statická analýza (Q1–Q3, Q5–Q7) a session metadata (E1–E3). Kvalitativní metriky (P2, Q4, Q8) hodnotí LLM-as-judge (sekce 3.2.4).
3. **Diagnose.** Identifikovat kde a proč agent nedodržel očekávané chování. Diagnóza postupuje ve čtyřech krocích: (1) *co selhalo*: component analysis (Mao et al., 2025) rozloží instrukce na složky a určí která z nich nedosáhla očekávaného efektu; (2) *proč to selhalo*:

script balance (Hassan et al., 2025) posoudí zda v instrukcích dominuje procedura na úkor kontextu, nebo naopak; (3) *stojí to za to*: content effectiveness (Lulla et al., 2026) vyhodnotí zda daná část instrukcí přidává hodnotu, nebo jen tokeny; (4) *jak opravit*: prompt sensitivity (Breunig, 2025; Razavi & Fard, 2025) říká, že opakování ignorované instrukce nepomáhá, účinnější je přestrukturování.

4. **Fix.** Upravit `AGENTS.md` s odkazem na diagnózu. Každá změna se zaznamená s citací a zdůvodněním.

Cyklus se opakuje dokud agent nesplní exit kritéria. Ta se dělí na deterministické (přirozená binární hranice) a minimální standard (kritérium zdůvodněné literaturou). Deterministická kritéria: P1 = 5/5, Q1 = match, Q2 = 40/40, Q4 = 24/24, Q5 = 0, Q6 = 0. Minimální standard: Q3 $\geq$ 70 % (práh zvolený autorem jako pragmatické minimum; Harman et al. (Harman et al., 2025) zjistili, že při plném coverage 70 % mutantů nezachytí ani produkční testy, takže nižší práh by nebyl smysluplnou hranicí), Q7 $\leq$ 10 per funkce (McCabe, 1976), P2 a Q8 $\geq$ 2/3 na škále LLM-as-judge. Metriky efektivity (E1–E3) se zaznamenávají bez kritéria pro srovnání mezi běhy. S ohledem na scope práce (jeden projekt, jeden model) je kritériem jeden úspěšný běh bez manuálního zásahu.

Každá iterace produkuje: aktualizovaný `AGENTS.md` (diff oproti předchozí verzi), záznam změn se zdůvodněním, kompletní P/Q/E metriky a diagnostickou zprávu. Konkrétní skripty a jejich konfigurace jsou v příloze.

3.3.4 Komparativní variace

[DRAFT]

Z fungující sady instrukcí (výstup pilotní fáze) systematicky měníme jednotlivé části a měříme dopad na chování agenta. Používáme dva typy změn: ablace (odebrání části, tedy funguje agent i bez ní?) a substituce (nahrazení alternativní formulací, tedy která varianta produkuje lepší výsledky?).

Konkrétní variace nelze specifikovat předem. Pilotní fáze *generuje* hypotézy o tom, které složky instrukcí jsou pro chování agenta podstatné, a komparativní fáze je testuje. Výběr variací bude vycházet z diagnostiky pilotních běhů pomocí rámců popsaných v sekci 3.3.3. Například pokud se v pilotu ukáže, že agent konzistentně ignoruje pravidlo o test-first commitech, komparativní variace může testovat, zda odebrání tohoto pravidla změní i ostatní metriky, nebo zda alternativní formulace vede k lepšímu dodržování.

S jedním během per variaci nelze statisticky odlišit efekt změny od šumu způsobeného nedeterminismem LLM (sekce 3.5). Výsledky komparativní fáze proto interpretujeme jako indikativní, ne kauzální.

3.4 Přehledová tabulka

[DRAFT]

Tabulka 3.1 shrnuje všechny metriky na jednom místě. Sloupec *typ* rozlišuje deterministická exit kritéria (přirozená binární hranice), minimální standard (práh zdůvodněný literaturou) a záznamové metriky (bez kritéria, slouží k porovnání mezi běhy). Automatizované metriky (P1, Q1–Q3, Q5–Q7, E1–E3) sbírá jeden skript; kvalitativní metriky (P2, Q4, Q8) hodnotí LLM-as-judge (sekce 3.2.4).

Tabulka 3.1: Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ

Kód	**Metrika**	**Nástroj**	**Exit kritérium**	**Typ**
Procesní (P) — jak agent pracuje				
P1	Process compliance (5 položek)	git, GitHub API	5/5	deter.
P2	Kvalita procesních výstupů	LLM-as-judge (GLM-5)	$\geq 2/3$	min.
Produktové (Q) — co agent vyrobil				
Q1	API contract match	tsc (import + typecheck)	match	deter.
Q2	Referenční test pass rate	Vitest (40 testů)	40/40	deter.
Q3	Mutation score	Stryker	$\geq 70\,\%$	min.
Q4	AC coverage (24 kritérií)	LLM-as-judge (GLM-5)	24/24	deter.
Q5	Lint warnings	ESLint	0	deter.
Q6	Typecheck errors	tsc `--noEmit`	0	deter.
Q7	Cyklomatická složitost	ESLint (complexity)	≤ 10/fn	min.
Q8	Design quality	LLM-as-judge (GLM-5)	$\geq 2/3$	min.
Efektivita (E) — za jakou cenu				
E1	Tokeny (input + output)	OpenCode export	—	záznam
E2	Trvání (minuty)	session timestamps	—	záznam
E3	Dokončení + restarts	auto-continue log	—	záznam

3.5 Omezení a validita

[DRAFT]

Každý výzkumný design má omezení. Následující analýza vychází z kategorizace Runeson a Höst (Runeson & Höst, 2009).

Konstruktová validita (měříme to co chceme měřit?). Jednotlivé metriky vychází z existující teorie: mutation testing (Papadakis et al., 2019), cyklomatická složitost (McCabe, 1976), taxonomie procesních a produktových metrik (Fenton & Bieman, 2014). Konkrétní kombinace metrik do sady a volba exit kritérií jsou autorské a představují návrh, ne ověřený standard. Tři metriky (P2, Q4, Q8) obsahují subjektivní složku (LLM-as-judge), kterou validujeme shodou s manuálním hodnocením (Cohenovo κ). Referenční testy (Q2) jsou odvozeny ze specifikace

metodou TDD, čímž se snižuje riziko test oracle problému (Mathews & Nagappan, 2024).

Interní validita (jsou závěry podložené daty?). Hlavní hrozba je nedeterminismus LLM: stejné instrukce mohou při různých bězích dát různé výsledky. S jedním během per variaci nelze statisticky odlišit efekt změny instrukcí od náhodného šumu (Razavi & Fard, 2025). Částečnou mitigací je více běhů v pilotní fázi. Další hrozbou je confirmation bias: autor navrhuje instrukce i evaluuje výsledky. Tuto hrozbu oslabují automatizované metriky (P1, Q1–Q3, Q5–Q7), které nepodléhají subjektivnímu posouzení, a volba judge modelu z jiné rodiny.

Externí validita (lze zobecnit?). Případová studie na jednom projektu, jednom modelu a jednom agentním nástroji neumožňuje statistickou generalizaci. Yin (R. K. Yin, 2018) pro tento typ výzkumu rozlišuje analytickou generalizaci: z jednoho případu lze ukázat principy, ne statistické zákonitosti. Systém upomínek má deterministickou logiku a výsledky nemusí platit pro projekty s uživatelským rozhraním, strojovým učením nebo nedeterministickými výstupy. Přenositelné jsou metriky a postup (kdokoliv je může použít na svém projektu), konkrétní naměřené hodnoty a instrukce platí pro tuto studii.

Výzkum nezahrnuje lidské účastníky. Veškerá data (git log, metriky, session transcripts) jsou generována experimentální infrastrukturou. Zdrojový kód a experimentální data jsou veřejně dostupná.

4. Praktická část

[DRAFT]

Tato kapitola popisuje provedení případové studie. Struktura sleduje chronologii experimentu: nejprve příprava tří výchozích podkladů (specifikace, referenční implementace, instrukce), pak pilotní iterace kde se instrukce opakovaně upravují na základě naměřených metrik, a nakonec komparativní variace kde z fungující sady systematicky odebíráme nebo nahrazujeme jednotlivé části.

4.1 Příprava experimentu

[DRAFT]

Před spuštěním pilotních běhů bylo nutné připravit tři výchozí podklady každého experimentálního běhu: specifikaci projektu (co má agent implementovat), referenční implementaci (strop kvality pro měření) a baseline instrukce (jak má agent pracovat).

4.1.1 Konstrukce specifikace

[DRAFT]

Specifikace definuje *co* má agent implementovat. Má formu strukturované GitHub issue (Issue #1, kterou agent dostane jako zadání) a skládá se ze dvou částí: **Requirements** (co business potřebuje) a **API Contract** (co musí implementace exportovat). Agent nedostává architektonický návrh (behavioral model, state diagram, invarianty) — architekturu navrhuje sám na základě požadavků. API kontrakt (exportované funkce a typy) je jediné technické omezení a slouží jako vstup pro metriku **Q1**. Specifikace je záměrně stručná: recall modelu klesá s délkou kontextu (**liu2024lost**) a redundantní sekce vedou k nekonzistencím (Bockeler, 2025).

Specifikace obsahuje:

- **24 acceptance criteria** ve formátu Given/When/Then s konkrétními hodnotami — eskalační flow (8 stavových přechodů), platby, terminální stavy, pause/resume, manuální advance, konfigurovatelné timeouty, výpočet pracovních dní
- **API kontrakt** v TypeScriptu — veřejná funkce `process(state, event, now)` vracející nový stav a action descriptory; typy pro 12 stavů, 6 událostí, 3 typy akcí; konfigurační interface pro timeouty a svátky
- **Doménový slovník** — 9 pojmů definujících business doménu (dunning, grace period, business days, action descriptor aj.)

- **Out of scope** — 6 explicitně vyloučených oblastí (payment retry, late fees, partial payments, email sending, scheduling, persistence)

Formát acceptance criteria přímo ovlivňuje měřitelnost. Given/When/Then struktura umožňuje přímý překlad do testů, takže agent nemusí interpretovat nestrukturovaný text a variabilita mezi běhy se snižuje. Referenční testy (metrika **Q2**) i mapování AC na agentovy testy (metrika **Q4**) z tohoto formátu přímo vychází.

Specifikace definuje systém jako pure function: `process(state, event, now)` vrací nový stav a action descriptory místo provádění side effects. Toto rozhodnutí přímo umožňuje automatizované měření — referenční testy (metrika **Q2**) i mutation testing (metrika **Q3**) vyžadují deterministické vstupy a výstupy bez infrastrukturních závislostí. Pevné TypeScript signatury (API kontrakt) zajišťují, že testy lze spustit na libovolném agentním běhu nezávisle na interní struktuře kódu.

Specifikace obsahuje explicitní sekci out of scope, která vymezuje 6 oblastí mimo zadání. Bez tohoto vymezení hrozí, že agent přidává funkcionalitu nad rámec zadání.

4.1.2 Referenční implementace

[DRAFT]

Referenční implementace slouží jako měřicí nástroj: definuje strop kvality proti kterému se porovnávají agentovy běhy. Její role v metrikách popisuje sekce 3.2.2.

Testy byly napsány metodou *spec-first TDD* (sekce 2.4.2): expected values pochází ze specifikace, ne z pozorování kódu.

Výsledná implementace obsahuje 45 behavioral testů pokrývajících všech 24 acceptance criteria (některá kritéria vyžadují více testových případů pro hraniční situace), dosahuje mutation score 91,9 %. Kód kompiluje bez typecheck errors, neobsahuje žádné `any` a má 3 lint warnings (kosmetické, bez vlivu na funkční korektnost). LLM-as-judge (**Q8**) ohodnotil design quality na 3/3 ve všech dimenzích. Tyto výsledky validují, že specifikace je implementovatelná a že exit kritéria (sekce 3.3.3) jsou dosažitelná lidským vývojářem.

4.1.3 Konstrukce baseline instrukcí

[DRAFT]

Baseline verze `AGENTS.md` byla konstruována čistě z literatury. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních.

Konstrukce proběhla ve třech krocích: smazání předchozí ad-hoc verze, návrh struktury podle empiricky odvozeného pořadí komponent Mao et al. (Mao et al., 2025) (Role → Goal →

Specification → Environment → Process → Package Quality → Constraints) a mapování tří dimenzí chování (sekce 3.2) na konkrétní instrukce.

Pořadí sekcí vychází z analýzy 2 163 produkčních šablon: Mao et al. (Mao et al., 2025) zjistili, že instrukce Role a Directive na začátku dokumentu korelují s vyšší efektivitou agenta, zatímco Constraints patří na konec. Toto empiricky odvozené pořadí tvoří páteř baseline dokumentu.

Sekce Package Quality definuje očekávání na kvalitu kódu: modulární struktura (typy, business logika a veřejné API v oddělených souborech), striktní typový systém bez obcházení (`any`), JSDoc dokumentace na exportech a jasně definovaný vstupní bod (`index.ts` re-exportuje pouze veřejné API). Tyto požadavky přímo ovlivňují metriky **Q5**–**Q8**. Lulla et al. (Lulla et al., 2026) dokládají, že instrukce zaměřené na architekturu a konvence jsou nejúčinnějším obsahem (−28 % doby běhu), zatímco podrobné step-by-step popisy přínos nepřinášejí.

Procesní sekce obsahuje stručné instrukce pro spec-first TDD (Mathews & Nagappan, 2024), dekompozici do sub-issues, branch-per-issue a conventional commits. Li et al. (Li et al., 2025) ukazují, že procedurální guidance ("jak pracovat") je efektivnější než popisná dokumentace ("jak vypadá codebase"). Z procesní sekce byly odstraněny bash příklady, protože verbose step-by-step instrukce nejsou efektivnější než stručné (Lulla et al., 2026) a redundantní obsah zvyšuje tokeny bez benefitu (Gloaguen et al., 2025).

Sekce Constraints obsahuje explicitní zákazy: nekombinovat issues, nemodifikovat referenční testy, nepřepisovat git historii. Mao et al. (Mao et al., 2025) zjistili, že exclusion constraints (46 %) jsou nejefektivnějším typem omezení.

Výsledný dokument má 53 řádků a ∼350 slov. Kompletní znění viz přílohu C. Tabulka 4.1 ukazuje mapování jednotlivých sekcí na metriky.

Tabulka 4.1: Mapování sekcí baseline `AGENTS.md` na metriky

Cíl	**Sekce AGENTS.md**	**Metriky**
Spec-first	Specification, Process bod 2	P1 (bod 1)
Strukturovaný workflow	Process body 1–4	P1 (body 2–4)
TDD	Process bod 2, Constraints	P1 (body 3, 5)
Modulární kód	Package Quality	Q5, Q7, Q8
API kompatibilita	Specification (API Contract)	Q1
Funkční korektnost	Specification (AC)	Q2, Q4

Specifikace, referenční implementace a baseline instrukce tvoří výchozí bod experimentu. Následující sekce popisuje průběh pilotních iterací cyklem Run/Measure/Diagnose/Fix (sekce 3.3.3).

4.2 Pilotní iterace

[DRAFT]

Každá iterace sleduje cyklus Run/Measure/Diagnose/Fix (sekce 3.3.3). U prvního běhu (baseline) uvádíme kompletní tabulku metrik a podrobnou diagnostiku. U dalších běhů popisujeme pouze změny oproti předchozí iteraci; souhrnný přehled všech běhů obsahuje sekce 4.4.

4.2.1 Pilot-r1: baseline

[DRAFT]

První běh s baseline instrukcemi (příloha C). Agent Minimax přes OpenCode, jedna session bez restartů. Tabulka 4.2 shrnuje naměřené metriky.

Tabulka 4.2: Pilot-r1: naměřené metriky

Kód	**Metrika**	**Hodnota**	**Kritérium**	**Splněno?**
P1	Process compliance	2/5	5/5	×
P2	Kvalita procesních výstupů	2/3	$\geq$ 2/3	✓
Q1	API contract match	match	match	✓
Q2	Ref. test pass rate	34/45	45/45	×
Q3	Mutation score	84 %	$\geq$ 70 %	✓
Q4	AC coverage	23/24	24/24	×
Q5	Lint warnings	2	0	×
Q6	Typecheck errors	0	0	✓
Q7	Složitost kódu	2 viol.	$\leq$ 10/fn	×
Q8	Design quality	1/3	$\geq$ 2/3	×
E1	Tokeny (in + out)	143k	—	záznam
E2	Trvání	33 min	—	záznam
E3	Dokončení	ano, 0 restartů	—	záznam

Ze 10 metrik s exit kritériem splnil agent 4 (P2, Q1, Q3, Q6).

Agent vytvořil 11 GitHub issues a rozložil implementaci do 4 pull requestů, každý s vlastní branch. Zvolil modulární strukturu (4 soubory: `types.ts`, `businessDays.ts`, `dunning.ts`, `index.ts`), což odpovídá požadavkům sekce Package Quality v instrukcích. Napsal 59 vlastních testů, všechny procházejí.

Diagnostika nesplněných metrik. Tři procesní položky z pěti nebyly splněny: agent sloučil 11 issues do 4 branches místo jedné branch per issue (P1 bod 2), nepoužil separátní `test:` commity (P1 bod 3) a upravil jeden referenční test soubor (P1 bod 5). Tyto tři selhání sdílí společný vzorec — agent upřednostnil rychlost dokončení před dodržením předepsaného workflow.

Z referenčních testů selhalo 11 z 45 (Q2 = 34/45). Selhání se koncentrují v pause/resume logice a hraničních případech výpočtu pracovních dní. Jedno acceptance criterion (AC 21) nemá v agentově test suite odpovídající test (Q4 = 23/24). Dvě funkce překračují McCabe práh složitosti 10 (Q7). LLM-as-judge ohodnotil design quality na 1/3 (Q8) — naming, separace zodpovědností, idiomatický TypeScript a složitost dosáhly 3/3, ale dokumentace 1/3 (žádný JSDoc na veřejném API), a celkové skóre odpovídá minimu dimenzí.

Změna instrukcí pro r2: bude doplněna po spuštění dalšího běhu.

4.3 Komparativní variace

4.3.1 Ablace

4.3.2 Substituce

4.4 Souhrnné výsledky

5. Vyhodnocení a diskuse

5.1 Interpretace výsledků

5.2 Porovnání s literaturou

5.3 Limity a hrozby validity

5.4 Doporučení pro praxi

5.5 Náměty pro další výzkum

Závěr

Bibliografie

ACE-Bench: End-to-End Feature Development Benchmark [212 tasks from 16 repos; Claude 4 Sonnet + OpenHands: 7.5% on feature-level tasks]. (2025). https://openreview.net/forum?id=41xrZ3uGuI

Anthropic. (2025). *Effective Context Engineering for AI Agents* [Introduces "context rot" – model recall degrades with token count. Recommends minimal high-signal token sets]. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Bockeler, B. (2025). *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl* [Critical analysis of SDD tools. Spec-kit files "repetitive, both with each other, and with the code"]. https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html

Breunig, D. (2025). Don't Fight the Weights [Defines fighting-the-weights: when prompt instructions oppose trained model behavior; recognition signs and mitigation strategies]. https://www.dbreunig.com/2025/11/11/don-t-fight-the-weights.html

Cao, L., & Ramesh, B. (2008). Agile Requirements Engineering Practices: An Empirical Study [16 organizations, 7 agile RE practices identified]. *IEEE Software*, *25*(1), 60–67. https://doi.org/10.1109/MS.2008.1

FeatureBench: Benchmarking Agentic Coding for Complex Feature Development [Agents struggle when scope exceeds single-issue; feature-level tasks significantly harder]. (2026). *arXiv preprint arXiv:2602.10975*.

Fenton, N. E., & Bieman, J. (2014). *Software Metrics: A Rigorous and Practical Approach* (3. vyd.) [Process vs. product vs. resource metrics taxonomy; GQM framework]. CRC Press.

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd). Addison-Wesley.

Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2025). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? [LLM-generated context files reduce success rate; developer-written marginally help (+4%); context files increase cost 20%+; agents follow instructions but no performance gain; AGENTBENCH benchmark (138 instances, 12 repos)]. *arXiv preprint arXiv:2602.11988*. https://arxiv.org/abs/2602.11988

Gotel, O. C., & Finkelstein, A. C. (1994). An Analysis of the Requirements Traceability Problem. *Proceedings of IEEE International Conference on Requirements Engineering*, 94–101.

Harman, M., Mao, K., Moran, K., Tufano, R., Gligoric, M., Shi, A., & Havrilla, A. (2025). Mutation-Guided LLM-Based Test Generation at Meta: A White Paper [70% mutants unkilled despite full coverage; 73% engineer acceptance rate; deployed on 10,795 classes].

Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic Software Engineering: Foundational Pillars and a Research Roadmap [SASE framework, BriefingScript, MentorScript, SE4H/SE4A duality]. *arXiv preprint arXiv:2509.06216*. https://arxiv.org/abs/2509.06216

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research [Foundational DSR framework: build-evaluate cycle, 7 guidelines for design science research]. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Chen, J., Lin, J., Xiong, Y., Lu, J., Zhang, H., & Xie, T. (2025). Rethinking the Value of Agent-Generated Tests in Autonomous Code Repair [83.2% tasks same outcome regardless of test writing; tests serve as observational feedback]. *arXiv preprint arXiv:2505.21615*.

IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge* (Version 4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [2294 tasks from real GitHub repositories, de facto benchmark for AI coding agents]. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.06770

Li, X., Chen, W., Liu, Y., Zheng, S., Chen, X., He, Y., Li, Y., et al. (2025). SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks [Curated Skills +16.2pp; self-generated Skills -1.3pp; 2–3 focused modules optimal; comprehensive documentation hurts (-2.9pp); smaller model + Skills can exceed larger model without; 84 tasks, 11 domains, 7308 trajectories]. *arXiv preprint arXiv:2602.12670*. https://arxiv.org/abs/2602.12670

Lulla, K., Zhu, M., Kalliamvakou, E., & Mohylevskyy, Y. (2026). On the Impact of AGENTS.md Files on AI Coding Agents [124 PRs across 10 repos: -28% runtime, -20% output tokens with AGENTS.md; architectural info and coding conventions most effective]. *arXiv preprint arXiv:2601.20404*. https://arxiv.org/abs/2601.20404

Mao, Y., He, J., & Chen, C. (2025). From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications [Merged 4 frameworks (Google Cloud, Elavis Saravia, CRISPE, LangGPT); 7 component types; Directive 87% prevalence]. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. https://arxiv.org/abs/2504.02052

Mathews, N. S., & Nagappan, M. (2024). LLM-Based Test Generation as Bug Validation: Insights from Reproducing Real-World Bugs [68.1% generated test suites validate bugs instead of detecting them]. *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*. https://doi.org/10.1145/3664646.3664766

McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, *SE-2*(4), 308–320. https://doi.org/10.1109/TSE.1976.233837

METR. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [RCT: 16 developers, 246 tasks; AI tools slowed experienced developers by 19%; developers believed they were 24% faster]. *arXiv preprint arXiv:2507.09089*. https://arxiv.org/abs/2507.09089

Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations [Causal link: self-recognition → self-preference; linear corre-

lation; fine-tuning pushes recognition to 90%+]. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2404.13076

Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey [Mutation score stronger predictor of fault detection than structural coverage]. *Advances in Computers*, *112*, 275–378. https://doi.org/10.1016/bs.adcom.2018.03.015

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2008). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Piskala, D. B. (2026). *Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants* (tech. zpr.) (Technical report. Three levels: spec-first, spec-anchored, spec-as-source. SDD workflow: Specify → Plan → Implement → Validate). arXiv. https://arxiv.org/abs/2602.00180

Razavi, S. P., & Fard, F. H. (2025). What Did I Do Wrong? Quantifying LLMs' Sensitivity and Consistency to Prompt Engineering [Up to 76 accuracy points difference from subtle formatting changes in few-shot settings]. *Proceedings of NAACL*. https://arxiv.org/abs/2406.12334

Runeson, P., & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering*, *14*(2), 131–164. https://doi.org/10.1007/s10664-008-9102-8

Scacchi, W. (2002). Understanding the Requirements for Developing Open Source Software Systems [Software informalisms in OSS – issue trackers as de facto requirements]. *IEE Proceedings – Software*, *149*(1), 24–39. https://doi.org/10.1049/ip-sen:20020202

Scale AI. (2025). SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? [1865 human-verified tasks, adds explicit Requirements and Interface sections to issues]. *arXiv preprint arXiv:2509.16941*. https://arxiv.org/abs/2509.16941

Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt Engineering or Fine-Tuning: An Empirical Assessment of Large Language Models for Code. *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. https://arxiv.org/abs/2310.10508

Sommerville, I. (2016). *Software Engineering* (10th). Pearson.

Wei, Y., et al. (2025). SWE-EVO: Long-Horizon Software Evolution Tasks [Multi-step modifications spanning avg 21 files; GPT-5 + OpenHands only 21% vs 65% on single-issue SWE-Bench]. *arXiv preprint arXiv:2512.18470*. https://arxiv.org/abs/2512.18470

Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6. vyd.) [Embedded single-case design; analytic generalization (to theory, not population)]. SAGE Publications.

Yin, Z., Wang, J., Li, J., Shi, D., Wei, Y., Yue, Y., Liu, Z., Xia, X., & Lo, D. (2025). A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks. *arXiv preprint arXiv:2511.00872*.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Foundational LLM-as-judge paper; position bias,

verbosity bias, self-enhancement bias; GPT-4 >80% human agreement]. *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.05685

Přílohy

A. Formulář v plném znění

B. Zdrojové kódy výpočetních procedur

C. Baseline AGENTS.md

Níže je úplné znění baseline verze instrukčního dokumentu `AGENTS.md`, konstruované čistě z literatury (sekce 4.1.3). Dokument má 7 sekcí, 53 řádků a ∼350 slov.

```
# AGENTS.md --- Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm
package. You prioritize clean architecture, spec compliance, and test
quality.

## Goal

Implement the dunning system (billing reminder state machine)
   specified
in GitHub Issue #1. Deliver a modular, documented, publishable
TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- **Acceptance Criteria** --- every one must be satisfied and tested
- **API Contract** --- exact TypeScript signatures to export
- **Domain Glossary** --- use these terms in code and documentation
- **Out of Scope** --- do not implement

All design decisions must trace back to the spec. Do not invent
requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code.
Work through them sequentially:

1. Create issues --- one concern per issue, starting with project
   setup
2. For each issue: branch from main, write tests first, implement,
   PR with "Closes #N", merge
```

```
3. Use conventional commits: ‘test:‘ for tests, ‘feat:‘ for
   implementation, ‘docs:‘ for documentation
4. Run ‘tsc --noEmit‘ before every PR --- no type errors allowed

## Package Quality

Structure the code as a production-grade npm package:

- **Modular architecture** --- separate files for types, business
  logic, utilities, and public API re-exports. No single-file
  implementations.
- **Strict TypeScript** --- no ‘any‘, explicit return types on public
  API, ‘readonly‘ where applicable.
- **Documentation** --- JSDoc on all exported functions and types.
  Inline comments for non-obvious domain logic (e.g., business day
  calculations, escalation thresholds).
- **Clean API surface** --- ‘src/index.ts‘ re-exports only the public
  API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the
  spec --- fix the code, not the test.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest,
  typescript).
```

Listing C.1: Baseline AGENTS.md — vstupní instrukce pro pilotní iteraci